从Claude Code刚泄漏的源码，看当前第一梯队 AI Agent 的工程架构

Max For AI
@MaxForAI · 15h

Follow

3 24 163 43K

今天（2026年3月31日），Anthropic 再次因为打包流程的低级失误，将其最新版 Claude Code（v2.1.88）的完整前端与客户端源码暴露在了 npm 仓库中。

网友发布了一个未被剔除的 cli.js.map 文件，**直接还原出了约 1900 个文件、超过 51 万行的原生 TypeScript 代码。**

对于 Anthropic 而言，这是继前几天 Mythos 模型文档外泄后的又一次严重 OpSec事故。

但对于整个大模型应用层的开发者和行业研究者来说，**这份源码却是一份毫无保留的、价值极高的前沿 AI Agent 工程架构白皮书。**

抛开合规与泄露事件的争议，我花了一些时间在本地对这份源码进行了深度梳理。

如果不把它看作一个八卦，而是看作一个生产级 AI 编程助手架构案例，**里面有大量突破常规思维的工程决策。**

以下是我以客观视角，对 Claude Code 底层架构、调度机制、记忆系统及安全策略的详细技术拆解。

文章较长，适合从事 AI Infra、Agent 开发以及对大模型应用层架构感兴趣的从业者阅读。

不仅仅是一个 CLI 工具

从目录结构（src/ 下约 40 个一级模块）可以看出，Claude Code 的复杂度远超目前市面上开源的常规单体 Agent。

它的技术栈选型非常务实且注重终端交互体验：

语言为 TypeScript，运行时选择了性能更激进的 Bun，CLI 框架使用 Commander，而终端渲染层则出人意料地使用了 React + Ink。

为什么一个命令行工具要用 React?

源码中的 screens/REPL.tsx（高达 5005 行）给出了答案。

在大模型流式输出（Streaming）和多工具并发执行的场景下，终端 UI 的状态管理变得极其复杂（例如同时渲染思考过程、工具调用进度条、代码 Diff 预览等）。

采用声明式的 React 配合极简的 Zustand 风格自定义 Store（state/store.ts），是应对这种高频局部刷新的最佳工程实践。

在运行模式上，系统被严密地划分为两种形态：

交互式 REPL 模式：通过 Ink 驱动前端终端 UI，主要面向人类开发者。

无头/SDK 模式（QueryEngine 类）：完全剥离 UI，支持 JSON 流式输出。这为后续将其作为底层引擎嵌入 IDE（如类似 Cursor 的形态）或 CI/CD 流程中埋下了伏笔。

系统启动流程也做了极致的并发优化。

在 main.tsx 中，配置读取（MDM Settings）和 Keychain 密钥预取等 I/O 密集型操作被放在子进程中，与主模块 ~135ms 的加载过程并行执行，这种对启动延迟的毫秒级苛求，贯穿了整个代码库。

Prompt Cache（提示词缓存）工程学

这是整份源码中最具技术含量的部分，也是拉开 Claude Code 与普通套壳应用体验差距的核心壁垒。

目前 Agent 工具在处理长上下文时，往往还在简单粗暴地拼接 System Prompt 和历史对话。

而在 Claude Code 的 services/api/claude.ts（长达 3419 行的核心交互模块）中，提示词组装被做到了字节级的精打细算。

众所周知，Anthropic 的 Prompt Cache 机制采用前缀匹配（Prefix Matching）。

为了最大化缓存命中率，Claude Code 设计了严密的**分段缓存架构**：

静态段（全局可缓存）：通过 systemPromptSection() 生成，包含模型身份介绍（"You are Claude Code..."）、系统级安全规则、代码风格限制、工具使用基础指南等。这部分在整个会话生命周期内几乎不变。

动态分界线：源码中硬编码了一个特殊标记 SYSTEM_PROMPT_DYNAMIC_BOUNDARY。

动态段（会话级缓存/不缓存）：包含当前工作目录信息（CWD）、Git 状态、MCP（Model Context Protocol）指令、用户配置等高频变化的数据。

并且为了防止 Prompt 发生微小变化导致缓存穿透，系统做了大量看似繁琐的兜底工作：

- **确定性排序**：传给大模型的工具描述（Tools Description）被严格按照内置工具前缀 + MCP 工具后缀进行字母表排序。
- **哈希路径映射**：配置文件的路径不使用随机 UUID，而是使用基于内容的哈希值，避免每次注入路径不同破坏缓存。
- **状态外置**：甚至连当前可用的 Agent 列表，也被从工具描述中剥离，转移到了消息附件（Attachments）中。据源码注释透露，仅这一项改动就减少了约 10.2% 的 Cache Creation Tokens 消耗。

这一切都在说明一个行业现状：**现阶段优秀的 AI 应用层开发，本质上就是在贪婪且精细地压榨 API 缓存系统的价值。**

Tools与流式并发执行

Claude Code 内置了超过 40 种工具（涵盖文件读写、Bash 执行、网络抓取等），其工具系统架构采用了高度模块化的工厂模式（Factory Pattern）。

每个工具继承自基础的 Tool 接口，必须实现诸如 checkPermissions()、validateInput() 和 isConcurrencySafe()（是否并发安全）等方法。

按需加载的 ToolSearch 机制：当工具数量超过某个阈值时，如果把所有工具的描述都塞进 Prompt，Token 成本将不可接受。

源码中展示了一种名为 ToolSearch 的优雅策略：**非核心工具（如某些特定的分析插件）被标记为 defer_loading: true。**

模型在当前 Prompt 中看不到这些工具的具体定义，只知道有一个 ToolSearch 工具。当模型认为自己需要额外能力时，必须先调用 ToolSearch 去动态加载对应的工具配置。

StreamingToolExecutor（流式工具执行器）：为了提升执行效率，系统支持工具的并发调用。

协调器（toolOrchestration.ts）会将大模型返回的工具调用请求分区为并发批次和串行批次。

并发安全的工具（如同时读取多个不相关的文件、并发发起网络搜索）会被并行触发，而非并发安全的工具（如先后修改同一个代码文件）则严格串行。

大结果集的工具（如全盘 Grep 搜索）设有 maxResultSizeChars 预算，超过预算的内容会被直接截断并持久化到本地临时文件中，**只给 LLM 返回一个预览摘要，防止超大结果撑爆上下文窗口。**

解决上下文污染的Fork机制

目前的单体 Agent 存在一个致命缺陷：

在执行复杂任务（例如跨文件排查 Bug）时，模型可能会反复读取错误的文件、尝试错误的命令，这些试错过程会产生大量的垃圾上下文，**迅速污染主对话，导致模型在后续推理中精神分裂或遗忘初始目标。**

Claude Code 引入了复杂的 **协调器模式（Coordinator Mode）** 和 **Fork Subagent（派生子代理）** 机制来解决这一问题。

在环境变量启用协调器模式后，系统会被重构为 Coordinator-Workers 架构：

- **Coordinator（协调者）**：被剥夺了直接操作文件的权限，只保留 Agent（派生子代理）、SendMessage 和 TaskStop 三个工具。它的唯一工作是规划工作流（Research → Synthesis → Implementation → Verification）。
- **Workers（执行者）**：携带具体的工具描述被派生出来。

最值得称道的是其 **Fork 继承机制**。

当需要进行大范围代码探索时，Coordinator 会 Fork 出一个 Explore Agent。

这个子 Agent 会继承父对话的缓存（共享 Prompt Cache 以节约成本），但其后续的探索动作、读取的垃圾文件，完全在其隔离的上下文中进行。

探索结束后，**子 Agent 只需要通过特定的 XML 格式 <task-notification>，将提炼好的关键结论（Synthesis）传回给 Coordinator 的主上下文即可。**

这种用完即毁，只留结论的设计，是目前业界处理复杂多 Agent 长文本协同的最佳实践之一。

突破单体的 Agent Swarm并发机制

除了用于解决上下文污染的串行 Fork 机制，源码还展示了更具野心的并发多 Agent 架构——Swarm（Teammate）集群。

这部分逻辑主要隐藏在 utils/swarm/ 和 tasks/ 目录中。

系统支持一种名为 in_process_teammate 的任务类型。

在这种架构下，主进程可以平行唤醒多个 Agent（被称为 Teammate）同时执行不同的任务。

但在终端 CLI 环境中搞多 Agent 并发，会面临两个致命的工程挑战：权限弹窗冲突和 UI 渲染混乱。

Anthropic 的解法极为优雅：

- Leader 权限桥接（permissionSync.ts）：所有的 Teammate 子进程都不允许直接向用户弹窗请求权限。它们会将权限请求通过内部通道“桥接”给主进程的 Leader Agent，由 Leader 统一在主终端进行安全拦截和用户确认。
- 终端布局自动化：为了让用户能清晰地监控多个并行 Agent 的工作状态，源码直接集成了 iTerm2 和 Terminal.app 的 AppleScript 控制指令。当派生新的 Teammate 时，系统会自动在终端中切分窗格（Split Pane），为每个子 Agent 分配独立的输出视窗。

这标志着 AI 正在从“单体思考”正式向“集群并发协作”演进。

Dream（梦境）记忆架构

在 RAG（检索增强生成）大行其道的今天，几乎所有的 AI 产品都在集成向量数据库（Vector DB）。

但令人意外的是，Claude Code 的记忆系统（memdir/ 模块）极其复古且务实，它完全基于本地文件系统。

其架构由一个核心的 MEMORY.md（作为高层索引，被限制在最多 200 行/25KB 以内）和多个基于 Frontmatter 格式的主题文件组成。

记忆被精细划分为 User、Feedback、Project、Reference 四大类。

更有趣的是隐藏在源码中的 **KAIROS 助手模式**。

这是一个尚未正式发布的长期运行（Daemon）模式。

在 KAIROS 模式下，记忆系统不再是简单的索引更新，**而是采用了类似人类日志的追加模式（写入 logs/YYYY/MM/YYYY-MM-DD.md）。**

到了夜间或闲置时间，后台会唤醒一个名为 **Dream（做梦）** 的离线任务 Agent。

这个 Agent 的职责是对白天的流水账日志进行总结、蒸馏，然后将其提取固化到结构化的长期主题文件中。

这种从短期日志到长期记忆的异步整合机制，不仅绕开了向量检索的召回率痛点，还代表了端侧 AI 助理向永远在线、持续学习演进的明确方向。

权限收敛与安全

赋予 AI 执行本地 Shell 命令和修改文件的权限，是一把双刃剑。

频繁弹窗要求用户确认会彻底破坏自动化体验，而不加限制的自动执行则可能导致系统崩溃（如误执行 rm -rf）。

Claude Code 采用了一套多层权限收敛架构：

从底层的基于 @anthropic-ai/sandbox-runtime 的文件/网络沙箱，到特定危险操作（如 git push --force）的硬编码拦截，再到工具级别的校验。

但最引人注目的是其名为 **Auto Mode Classifier (yoloClassifier.ts)** 的组件。

当用户开启自动模式时，系统并没有使用死板的正则表达式来评估命令的危险性，而是使用了一个 **侧查询（Side Query）机制**。

系统会在后台静默调用一个更小、更便宜的 LLM，将当前对话的精简转录（Transcript）和即将执行的 Bash 命令抛给它，让这个侧边模型输出 Allow 或 Deny 的决策。 此外，系统内部还有一个基于阈值的Denial Tracking（拒绝追踪），当自动工具被频繁拒绝时，系统会优雅降级，退回到 Prompting 模式请求人类介入。

这种用小 AI 监管大 AI的动态权限系统，比传统的静态静态拦截规则要灵活得多。

一些小彩蛋

最后，源码中大量存在的 Feature Flags（如 VOICE_MODE、SSH_REMOTE 等）和 process.env.USER_TYPE === 'ant' 的环境变量判断，向我们展示了大厂在内部测试和外部发布时的双重标准。

对于 Anthropic 内部员工（Ant-only），系统注入的代码规范极其严厉甚至偏执：

不要擅自添加功能、如果要求没提就不要重构、三行相似的代码比过早的抽象更好、默认不写任何注释，除非 WHY 极不明显、测试失败了必须如实报告。

而对于外部公开构建，系统提示词则温和得多：直接切入主题，尝试最简单的方法，尽量简明扼要。

这种反差，说明大模型的行为边界很大程度上取决于硬编码的指令倾向。

值得注意的是代码里包含了两个有意思的模块。

卧底模式（Undercover Mode）： 这是一个备受安全社区争议的功能。

针对员工在开源或公共仓库工作的场景，系统默认开启且无法强制关闭该模式。该模式会在 Prompt 中明确要求模型Do not blow your cover（不要暴露身份），**并强制剥离所有由 AI 生成的免责声明或代号痕迹。**

从公关角度这或许显得缺乏透明度，但从侧面印证了厂商对模型角色扮演和输出干预的绝对控制力。

Buddy System（电子宠物）彩蛋： 源码中包含了一个隐藏的电子宠物系统（生成鸭子、猫头鹰等）。

为了保证宠物生成的随机性与确定性，工程师使用了用户的 ID 配合 Mulberry32 伪随机数生成算法。

```
typescript
// 18 种物种: duck, goose, blob, cat, dragon, octopus, owl, penguin, ...// 5 种稀有度: common(60%), uncommon(25%), rare(10%), epic(4%),
```

legendary(1%)// 属性: DEBUGGING, PATIENCE, CHAOS, WISDOM, SNARK// 配件: crown, tophat, propeller, halo, wizard, beanie, tinyduck// 特殊: 1% 概率 shiny

最搞笑的一个细节是，由于某个动物物种的英文名称恰好与 Anthropic 极其机密的内部模型代号重名（也许是前两天泄露的最强Claude卡皮巴拉）。

为了绕过合规代码扫描仪的违禁词检测，**工程师竟然使用了 String.fromCharCode() 来动态拼装这个单词。**

这种充满幽默感的极客做法，在极其严肃的基础设施代码中显得别具一格。

我们能学到什么？

在短时间内连续遭遇核心模型技术文档和核心应用源码的泄露，Anthropic 在内部流程管控上的确需要做深刻检讨。但技术无罪，这份 51 万行的代码对于行业而言是一份极好的教材。

从 Claude Code 的底层设计可以看出，**大模型应用层创业，单纯依靠拼凑 Prompt、堆砌向量数据库、套一个简单循环外壳的时代已经结束了。**

Want to publish your own Article?

Upgrade to Premium

干预的平衡（YOLO Classifier），以及对宿主操作系统深度的文件流集成上。

Max For AI
@MaxForAI

Follow

目前 GitHub 上 Fork 这些源码的仓库正面临随时被 DMCA 请求下架的风险。

但无论如何，Claude Code 展示出的工程化水平，已经为 2026 年的 AI Agent 产品树立了一个全新的技术标杆。

从业者们应当趁此机会，认真审视并吸纳其中的工程化最佳实践。